

student 테이블에는 학과(major)와 GPA가 있습니다. 학과별·GPA 구간별 인원을 집계하
되, 소계(소계 행)와 총계 행까지 한 번의 쿼리로 출력하세요.

```
SELECT
  CASE GROUPING(major) WHEN 1 THEN '전체' ELSE major END AS major,
  CASE GROUPING(gpa_tier) WHEN 1 THEN '소계' ELSE gpa_tier END AS gpa_tier,
  COUNT(*) AS cnt, AVG(gpa)::NUMERIC(4,2) AS avg_gpa
FROM (
  SELECT *, CASE WHEN gpa < 3.0 THEN 'C' WHEN gpa <= 3.5 THEN 'B' ELSE 'A' END
  AS gpa_tier
  FROM student
) t
GROUP BY ROLLUP(major, gpa_tier)
ORDER BY GROUPING(major), major, GROUPING(gpa_tier), gpa_tier;
```

enroll 테이블에는 고아 레코드(존재하지 않는 **student_id** 1001, 1010)가 포함되어 있습
니다. 수강 이력이 없는 학생 조회 시 **NOT IN** vs **NOT EXISTS**의 실행 계획 차이를 분석
하세요.

```
-- NOT IN (NULL 위험 예시)
SELECT * FROM student WHERE student_id NOT IN (SELECT student_id FROM enroll
WHERE student_id IS NOT NULL);
-- NOT EXISTS (권장)
SELECT * FROM student s WHERE NOT EXISTS (SELECT 1 FROM enroll e WHERE
e.student_id = s.student_id);
-- 성능 분석
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT) <위 쿼리>;
-- Hash Anti Join vs Nested Loop Anti Join 비교
```

emp 테이블은 CEO(manager_id = NULL) → 매니저(10명) → 개발자(300명)의 3단계 계층 구조입니다. **WITH RECURSIVE**로 모든 직원의 계층 경로와 깊이를 출력하세요.

```
WITH RECURSIVE org AS (  
    SELECT emp_id, name, manager_id, 0 AS depth, name::TEXT AS path  
    FROM emp WHERE manager_id IS NULL          -- Anchor: CEO  
    UNION ALL  
    SELECT e.emp_id, e.name, e.manager_id, o.depth + 1,  
           o.path || ' > ' || e.name  
    FROM emp e JOIN org o ON e.manager_id = o.emp_id  
)  
SELECT emp_id, name, depth, path FROM org ORDER BY path;
```

각 학과(major)별로 GPA가 높은 순서로 순위를 매기고, 상위 3명씩만 추출하는 쿼리를 **Window Function**으로 작성하세요. (서브쿼리 방식과 CTE 방식 모두 작성)

```
WITH ranked AS (  
    SELECT student_id, name, major, gpa,  
           ROW_NUMBER() OVER (PARTITION BY major ORDER BY gpa DESC, student_id)  
    AS rn,  
           RANK() OVER (PARTITION BY major ORDER BY gpa DESC) AS rnk,  
           DENSE_RANK() OVER (PARTITION BY major ORDER BY gpa DESC) AS drk,  
           COUNT(*) OVER (PARTITION BY major) AS total_in_major  
    FROM student  
)  
SELECT * FROM ranked WHERE rn <= 3 ORDER BY major, rn;
```

enroll 테이블을 **student_id** 기준으로 정렬하고, 각 학생의 이전 수강 과목 대비 성적 변화를 **LAG()**로 계산하세요. 성적은 A=4, B=3, C=2, D=1 점수로 변환합니다.

```
WITH scored AS (  
    SELECT student_id, course, grade,  
           CASE grade WHEN 'A' THEN 4 WHEN 'B' THEN 3 WHEN 'C' THEN 2 ELSE 1 END  
    AS score  
    FROM enroll  
)  
lagged AS (  
    SELECT *, LAG(score) OVER (PARTITION BY student_id ORDER BY course) AS  
    prev_score,  
           MAX(score) OVER (PARTITION BY student_id) - MIN(score) OVER (PARTITION BY  
    student_id) AS score_range  
    FROM scored  
)  
SELECT *, score - prev_score AS diff,  
       CASE WHEN score > prev_score THEN '상승' WHEN score = prev_score THEN '유지'  
    ELSE '하락' END AS trend  
FROM lagged;
```

orders 테이블의 주문을 **order_id** 순으로 정렬하여 누적 주문금액과 3개 주문 이동평균을 계산하세요. **ROWS BETWEEN** 절을 명시적으로 사용해야 합니다.

```
SELECT order_id, customer_id, amount,  
       SUM(amount) OVER (ORDER BY order_id ROWS BETWEEN UNBOUNDED  
PRECEDING AND CURRENT ROW) AS cum_total,  
       AVG(amount) OVER (ORDER BY order_id ROWS BETWEEN 2 PRECEDING AND  
CURRENT ROW) AS moving_avg_3,  
       SUM(amount) OVER (PARTITION BY customer_id ORDER BY order_id  
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT  
ROW) AS cust_cum  
FROM orders  
ORDER BY order_id;  
-- 50% 초과 첫 order: WHERE cum_total >= total_sum * 0.5 (CTE로 total 먼저 계산)
```